

Experiment 2: Fixed-Point Robotic-Arm Pick-and-Place

Individual Experimental Report

**Isaac Sim mechArm Simulation and DJI RoboMaster EP
Verification**

Course:	Robotic Systems Integration Laboratory
Experiment:	Fixed-Point Robotic-Arm Pick-and-Place
Report Type:	Individual Report
Name:	Haoran Xiao
Student ID:	24020036066
Project Role:	Simulation, control integration, debugging, physical verification, and documentation
GitHub:	xhr-CHN/mecharm-fixed-point-pick-place
Period:	August 31–September 9, 2026

September 9, 2026

Abstract

This individual report describes my work on Experiment 2, a fixed-point robotic pick-and-place task completed in both simulation and on a physical robot. The available platforms were different, so I used a mechArm 270 Pi with an adaptive gripper in Isaac Sim 5.1.0 for the simulation stage and a DJI RoboMaster EP for the physical stage. The course did not provide the specified physical mechArm platform, and the instructor was informed and approved this DJI EP substitution. In simulation, I built the scene, integrated the official Elephant Robotics model, connected Windows Isaac Sim to ROS 2 Humble in Docker through a TCP bridge, implemented numerical inverse kinematics and smooth direct joint execution, repaired the adaptive gripper's missing closed-loop pin constraints, and completed a five-cycle alternating transfer between two fixed points. For physical verification, I implemented and tested a five-cycle RoboMaster EP program with arm motion, chassis rotation, gripper-state feedback, automatic retry, camera monitoring, and a safe-exit key.

The project required substantial diagnosis across robot modeling, middleware discovery, container networking, trajectory generation, inverse-kinematics reachability, and contact dynamics. I therefore focus not only on the final workflow but also on the reasoning that connected each symptom to its root cause. Public GitHub evidence contains 31 pushed project commits authored by my account, while the local branch contains two additional report-planning commits. Simulation and physical recordings preserve both successful repeated operation and failure behavior. The final result demonstrates a reproducible fixed-point manipulation workflow and, more importantly, a systematic method for integrating software, simulation physics, and real hardware.

Keywords: fixed-point grasping; mechArm 270 Pi; Isaac Sim; ROS 2 Humble; inverse kinematics; adaptive gripper; RoboMaster EP; trajectory smoothing

Contents

Abstract	I
Chapter 1 Basic Experimental Requirements	1
1.1 Experimental Objective	1
1.2 Platforms Used in the Completed Experiment	1
1.3 Functional and Safety Requirements	2
1.4 Evaluation Criteria I Used	3
Chapter 2 Individual Responsibilities and Contributions	3
2.1 Scope of My Work	3
2.2 Git History as Verifiable Evidence	3
2.3 My Contribution Breakdown	4
2.4 Development Timeline	5
Chapter 3 Principles and Project Implementation	6
3.1 Overall System Architecture	6
3.2 Scene and Official Robot Model	6
3.3 Fixed Cartesian Targets	7
3.4 Numerical Inverse Kinematics and Grasp Orientation	8
3.5 Smooth Joint Execution	8
3.6 Adaptive-Gripper Four-Bar Repair	8
3.7 Five-Cycle Alternating Simulation	9
3.8 Empty-Grasp and Object Retention Logic	10
3.9 Physical Implementation on DJI RoboMaster EP	11
Chapter 4 Problems, Solutions, and Personal Reflection	12
4.1 Why I Focused on Root Causes	12
4.2 ROS 2 Discovery and Blank RViz	12
4.3 Docker Build and Proxy Failures	12
4.4 Motion Jerk, Shaking, and Over-Damping	13
4.5 Incorrect Grasp Position and Orientation	13
4.6 Missing Gripper Pins and Mimic-Joint Asymmetry	14
4.7 Failed Attempts as Engineering Evidence	15
4.8 Summary of Major Problems and Decisions	15
4.9 Personal Growth	16
Chapter 5 Conclusion	17
A Repository Evidence Cross-Reference	17
B Reproducible Operating Commands	18
B.1 Simulation Startup	18
B.2 Physical-Robot Startup	19
C Evidence Integrity Note	19

Chapter 1 Basic Experimental Requirements

1.1 Experimental Objective

The experiment required a robot to transfer an object between two known locations without visual localization. Because both points were predetermined, the essential engineering problem was not object detection but reliable motion execution: the robot had to start from a safe HOME posture, approach the object, align the gripper, close it without collision, lift the object, move to the destination, release it, and return safely.

I summarized the required state sequence as

HOME → PREGRASP → DESCEND → CLOSE → LIFT → PLACE → OPEN → HOME. (1)

The experiment also required repeated execution, visible state or error information, and safe behavior when a target was unreachable or a grasp failed.

1.2 Platforms Used in the Completed Experiment

The available equipment determined the implementation platform for each stage. I completed the simulation using the mechArm 270 Pi model and completed the physical stage using a DJI RoboMaster EP. The instructor was informed of the equipment shortage and approved the EP as the physical platform. This was a practical platform adaptation rather than an incomplete substitution: the two platforms demonstrated the same manipulation concept through their own suitable motion interfaces.

Table 1 Completed experiment stages and platforms

Stage	Platform	Main verification target	Status
Simulation	mechArm 270 Pi adaptive-gripper model in Isaac Sim 5.1.0	Cartesian approach, vertical grasp, A/B transfer, five alternating cycles, exceptions, and physics stability	Completed
Physical robot	DJI RoboMaster EP with robotic arm, gripper, chassis, and camera	Real grasp, transport through chassis rotation, release, automatic retry, safe exit, and five successful cycles	Completed

The simulation host was Windows, while ROS 2 Humble and MoveIt 2 ran in a dedi-

cated Ubuntu 22.04 Docker image. Isaac Sim and the container exchanged commands and measured joint states through a project-specific TCP bridge on port 8765. The physical RoboMaster program ran from Windows with the official RoboMaster Python SDK and connected to the robot in AP mode over TCP.

1.3 Functional and Safety Requirements

I translated the basic requirements into testable engineering items before implementation. The resulting checklist is shown in Table 2.

Table 2 Requirement-to-evidence matrix

Requirement	Implemented evidence	Result
Known pick and place locations	Parameterized A and B coordinates in the ROS 2 Cartesian configuration	Completed
Safe initial state	Defined HOME pose; every simulation cycle and every physical attempt begins and ends in a safe posture	Completed
Complete pick-and-place sequence	Open, approach, descend, close, lift, transfer, release, retreat, and HOME	Completed
Vertical grasp in simulation	Tool-axis constraint and a grasp-phase yaw offset used consistently through grasp and transport	Completed
Repeated operation	Five alternating A-to-B/B-to-A simulation cycles and five successful RoboMaster EP physical cycles	Completed
Failure handling	IK rejection, stale-feedback timeout, empty-grasp test, RoboMaster unknown-state timeout, retry, and safe exit	Completed
Observable evidence	GitHub history, terminal logs, simulation recordings, a failure recording, and a physical recording	Completed
Reproducible startup	PowerShell, Docker Compose, ROS 2 launch, scene rebuild, and physical SDK commands documented	Completed

1.4 Evaluation Criteria I Used

I considered the experiment successful only when the following conditions were visible together:

1. the robot started from a predictable HOME or parked posture;
2. the end effector reached a collision-free position above the target;
3. the gripper orientation and closing height matched the object's geometry;
4. the object was lifted before horizontal transport;
5. the object was released at the opposite point;
6. the robot returned to its initial posture before the next cycle;
7. failures produced a clear message and a bounded recovery rather than uncontrolled motion.

These criteria shaped both my code structure and my debugging decisions.

Chapter 2 Individual Responsibilities and Contributions

2.1 Scope of My Work

My work covered the full path from an empty repository structure to recorded simulation and physical-robot demonstrations. I organized the workspace, created the public repository, integrated the model, implemented the control programs, diagnosed middleware and physics failures, tuned the task parameters, wrote the startup documentation, recorded the experiments, and prepared the final reports.

I did not treat these as unrelated tasks. For example, a gripper-model defect directly changed the feasible trajectory and contact behavior; network discovery affected whether RViz and MoveIt appeared functional; and the hardware platform changed the available feedback signal for grasp detection. My responsibility was therefore system integration across these boundaries.

2.2 Git History as Verifiable Evidence

At the time of evidence capture, the public origin/main branch contained 31 commits, all authored by the GitHub account xhr-CHN. The local main branch contained 33 commits because it also included the individual-report design and implementation-plan commits. Figure 1 is a direct screenshot of the public GitHub commit page. It shows a representative continuous sequence of my feature, fix, tuning, and documentation commits.

xhr-CHN committed 17 hours ago	
docs: design low-speed S-curve motion	381ce4b
xhr-CHN committed yesterday	
feat: add retry and multi-cycle EP grasp scripts	4e6f3d2
xhr-CHN committed yesterday	
feat: organize RoboMaster EP control scripts	f2458ac
xhr-CHN committed yesterday	
Commits on Sep 7, 2026	
tune: sync grasp pose and Isaac drive parameters	5140448
xhr-CHN committed 2 days ago	
tune: raise pregrasp clearance before closing	ac41e48
xhr-CHN committed 2 days ago	
fix: stabilize adaptive gripper loop physics and refresh scene	8220502
xhr-CHN committed 2 days ago	
Commits on Sep 1, 2026	
docs: document reproducible direct grasp workflow	5943a0b
xhr-CHN committed last week	
feat: add calibrated direct Cartesian grasping	c5863e9
xhr-CHN committed last week	
feat: add containerized Isaac TCP runtime	0026efe
xhr-CHN committed last week	
docs: design parallel gripper evaluation	d5a2ef3
xhr-CHN committed last week	
fix: enable end-to-end pick-and-place transport	fd7be61
xhr-CHN committed last week	
fix: execute Isaac ROS graph every physics step	f4d5e93
xhr-CHN committed last week	
feat: add collision-aware pick and place workflow	2a51a7f
xhr-CHN committed last week	
feat: bridge Movelt trajectories to Isaac Sim	—

Figure 1 Public GitHub commit-history excerpt showing implementation, repair, tuning, and documentation commits authored by xhr-CHN

I use this history only as evidence of my own sustained work. The strongest contribution evidence is the combination of commit history, corresponding source files, runtime logs, and recorded behavior.

2.3 My Contribution Breakdown

Table 3 Personal contribution breakdown

Area	Work I completed	Representative evidence
Repository engineering	Created the repository structure, staged commits in logical increments, maintained main, added bilingual README files, and organized large videos with Git LFS	Git history and README files
Robot model	Integrated Elephant Robotics mechArm 270 Pi and adaptive-gripper URDF/meshes, generated the USD scene, and retained the source license	simulation/urdf/ and USD scene
Simulation scene	Added table, block, lighting, camera, fixed points, safe height, articulation setup, physics step, and render settings	Figure 3 and scene generator

Area	Work I completed	Representative evidence
ROS 2 integration	Built ROS 2 packages, MoveIt configuration, task nodes, launch files, state publishing, and trajectory interfaces	src/ packages and launch files
Cross-system transport	Replaced unreliable cross-OS DDS discovery with a narrow TCP bridge carrying joint state and targets	Port 8765 bridge and probe logs
Kinematics	Implemented numerical IK with position, finite-value, joint-limit, and vertical-axis checks	numeric_ik.py
Motion quality	Implemented quintic interpolation, 120 Hz target streaming, separate normal/vertical speed limits, feedback pacing, and timeout handling	Direct motion programs
Gripper mechanics	Diagnosed missing closed-loop pins, repaired the two distal loop constraints, added visible pins, disabled unreliable mimic parsing, and mirrored inner-joint commands	Figures 4 and 5
Repeated simulation	Implemented A-to-B/B-to-A alternation for five cycles, returning HOME after every transfer	Figure 6
Exception behavior	Implemented unreachable-IK rejection, stale-feedback timeout, and an intentional empty-grasp test with HOME recovery	Failure recording and logs
Physical verification	Implemented connection tests, small-motion tests, one-cycle and five-cycle EP programs, camera display, feedback-based grasp inference, retry, and E-key safe exit	Figure 7
Documentation	Wrote Chinese and English instructions, model source notes, test records, and experiment reports	README.md, README_EN.md, docs/, and report/

2.4 Development Timeline

My commit history also records how the system evolved:

- **August 31:** repository structure, experiment overview, simulation configuration, ROS 2 package metadata, Isaac integration, model assets, and the first smooth in-app demonstration;
- **September 1:** MoveIt configuration, trajectory bridge, collision-aware workflow, Isaac graph stepping, end-to-end transport repair, and containerized TCP runtime;

- **September 2:** calibrated Cartesian grasping and reproducible direct-grasp documentation;
- **September 7:** adaptive-gripper loop repair, pre-grasp clearance correction, pose calibration, and drive-parameter synchronization;
- **September 8:** RoboMaster EP programs, retry logic, five-cycle recordings, and bilingual final documentation;
- **September 9:** evidence-driven individual report design, implementation, and verification.

This sequence shows that I first established basic movement, then solved integration and physical-model defects, and only then finalized repeated operation and documentation.

Chapter 3 Principles and Project Implementation

3.1 Overall System Architecture

The main simulation challenge was that Isaac Sim ran natively on Windows while ROS 2 and MoveIt ran in Docker. I separated planning/control from physics, with the TCP bridge as a narrow transport boundary.

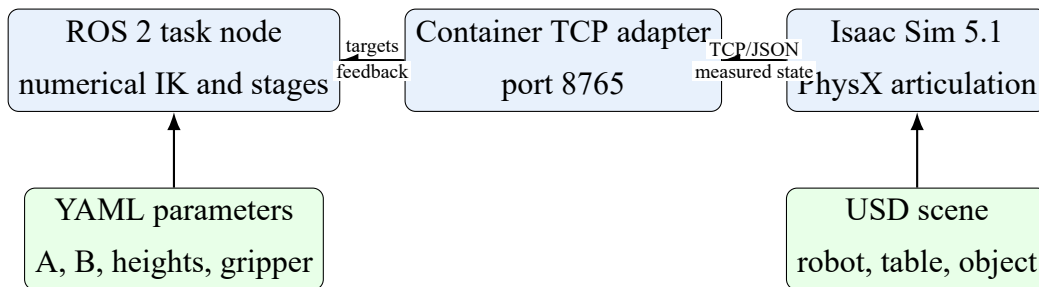


Figure 2 Simulation control architecture implemented for Experiment 2

This architecture kept the ROS graph container-local and removed the need for DDS multicast to cross Windows, WSL, Docker, and proxy/TUN boundaries. The transport sent only the state needed by this experiment, which made failures easier to isolate.

3.2 Scene and Official Robot Model

I based the robot description on the Humble branch of Elephant Robotics’ official mycobot_ros2 repository[2]. The imported assets include the mechArm 270 Pi arm and adaptive gripper meshes. The experiment scene contains a fixed-base robot, table, target block, camera, lights, and configured pick/place locations.

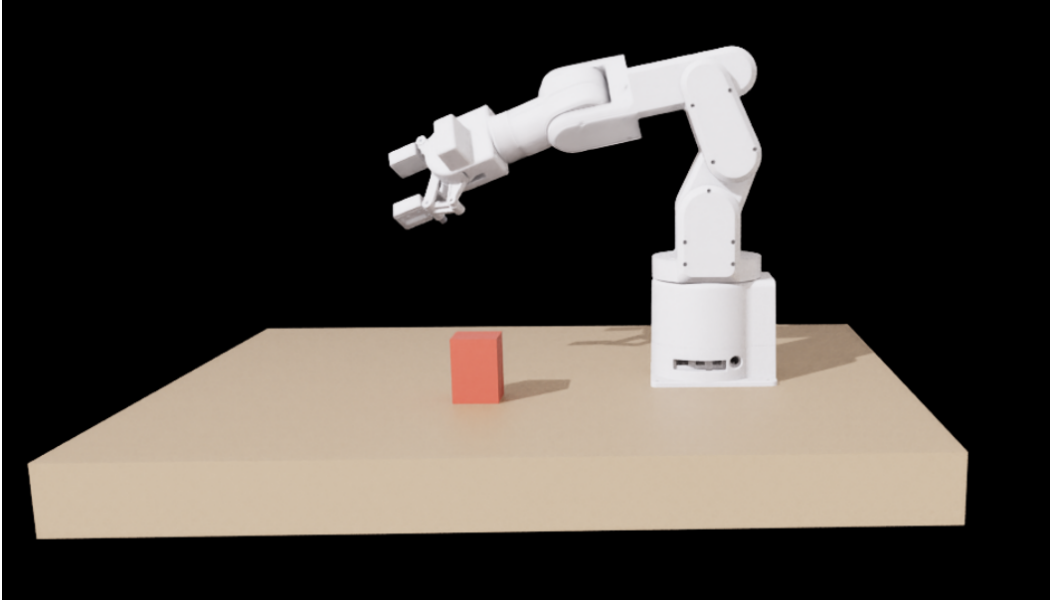


Figure 3 Completed Isaac Sim scene with mechArm 270 Pi, table, adaptive gripper, and target block

The physics world runs at 120 Hz and renders at 60 Hz. The checked-in scene generator imports the arm with default drive strength 1000 and damping 30000. The adaptive-gripper driven joints are then tuned separately to stiffness 150, damping 60, and maximum force 10. This separation was important because arm holding strength and closed-loop gripper stability require different drive behavior.

3.3 Fixed Cartesian Targets

The final simulation parameters were

$$\mathbf{p}_A = (0.18, 0.09, 0.025) \text{ m}, \quad (2)$$

$$\mathbf{p}_B = (0.18, -0.08, 0.025) \text{ m}. \quad (3)$$

The pre-grasp clearance is 0.10 m, so the first Cartesian target above the object is at approximately $z = 0.125$ m. The closing clearance is 0.04 m, giving a closing target at approximately $z = 0.065$ m. Since the top of the block is near $z = 0.050$ m, the fingers can surround the block without the palm colliding with it before closure.

The HOME joint pose is

$$\mathbf{q}_{\text{home}} = (0, 0, 0, -47, 0, 0)^\circ. \quad (4)$$

Every cycle returns to this pose before the next cycle starts.

3.4 Numerical Inverse Kinematics and Grasp Orientation

The numerical IK solver parses the same URDF that defines the simulated robot. For a candidate joint vector $\mathbf{q}$, forward kinematics produces the tool position $\mathbf{p}(\mathbf{q})$ and tool axis $\mathbf{a}(\mathbf{q})$. I accepted a solution only when

$$\|\mathbf{p}(\mathbf{q}) - \mathbf{p}_d\| \leq 0.008 \text{ m}, \quad (5)$$

$$\mathbf{a}(\mathbf{q}) \cdot \mathbf{a}_d \geq 0.975, \quad (6)$$

and all joint values were finite and within URDF limits.

The end effector also uses a 90° yaw offset relative to the HOME tool direction during the grasp phase. A key design decision was to carry the same orientation through pre-grasp, descent, lift, transport, placement, and retreat. Changing the yaw only at one waypoint would create unnecessary wrist motion and could turn a valid grasp into a collision.

3.5 Smooth Joint Execution

The first direct commands caused visible jerk and excited the compliant dynamics. I replaced piecewise target jumps with a quintic smoothstep:

$$s(u) = 6u^5 - 15u^4 + 10u^3, \quad 0 \leq u \leq 1, \quad (7)$$

$$\mathbf{q}(u) = \mathbf{q}_0 + s(u)(\mathbf{q}_1 - \mathbf{q}_0). \quad (8)$$

This polynomial has zero velocity and zero acceleration at both endpoints. Targets are streamed at 120 Hz and paced using measured feedback. If valid feedback is absent for more than 5 s, the program stops the segment instead of continuing open-loop.

I used two speed limits because vertical contact motion needs more control than free-space transit:

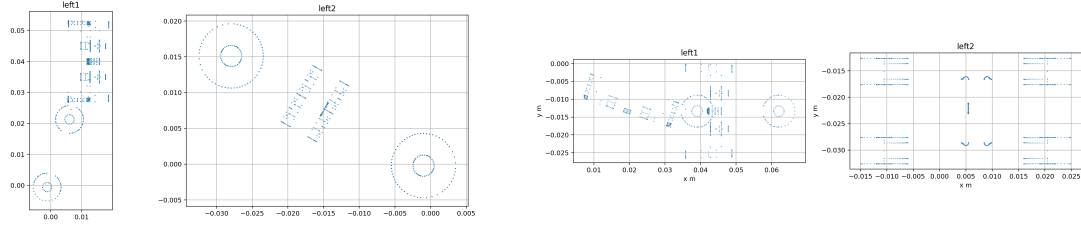
- normal arm stages: $28^\circ/\text{s}$ maximum peak joint speed;
- vertical descent/lift stages: $4^\circ/\text{s}$ maximum peak joint speed;
- adaptive-gripper command: 0.12 rad/s with 1.0 s pre-command and 1.5 s post-command waits.

This preserved reasonable cycle time while keeping the grasp approach slow.

3.6 Adaptive-Gripper Four-Bar Repair

The adaptive gripper was the most important model-level problem. The official URDF is a tree, but the real gripper contains closed four-bar linkages. Two distal pin joints that physically connect the inner and outer links are therefore not expressible as ordinary URDF

tree joints. The imported model visually contained pivot holes but lacked the two physical loop closures.



(a) Projected locations of the missing loop closures

(b) Linkage geometry used to align the pivots

Figure 4 Geometric diagnosis of the adaptive-gripper linkage

I repaired the mechanism in four steps:

1. disabled URDF mimic parsing because PhysX 5.1 did not reliably support multiple followers of the same reference joint;
2. added explicit angular drives to the inner links and mirrored the master command in the TCP joint bridge;
3. made the outer jaws passive and added two PhysX revolute loop joints between the inner and outer distal links;
4. added four visible, non-colliding cylinder pins so the physical constraints also had a clear visual representation.

The mirrored command relationship is

$$q_L = q_m, \quad q_{R2} = q_{R3} = -q_m, \quad (9)$$

while the passive outer jaws follow through the loop constraints. The scene uses 64 position and 64 velocity solver iterations and selective self-collision to improve stability.

Implemented pin model. The scene generator creates two excluded-from-articulation PhysX revolute joints named `gripper_left_loop_joint` and `gripper_right_loop_joint`. Four visible pins are placed at the base and distal pivots. The distal pair closes the two four-bar loops; the base pair represents the already constrained pivots. Pin meshes are visual only and do not create duplicate collisions.

Figure 5 Logical structure of the repaired gripper constraints

3.7 Five-Cycle Alternating Simulation

The repeated task alternates the source and destination:

$$A \rightarrow B, \quad B \rightarrow A, \quad A \rightarrow B, \quad B \rightarrow A, \quad A \rightarrow B. \quad (10)$$

After each placement, the robot returns HOME before beginning the next grasp. This makes each cycle independently observable and prevents pose drift from being hidden inside a long uninterrupted trajectory.

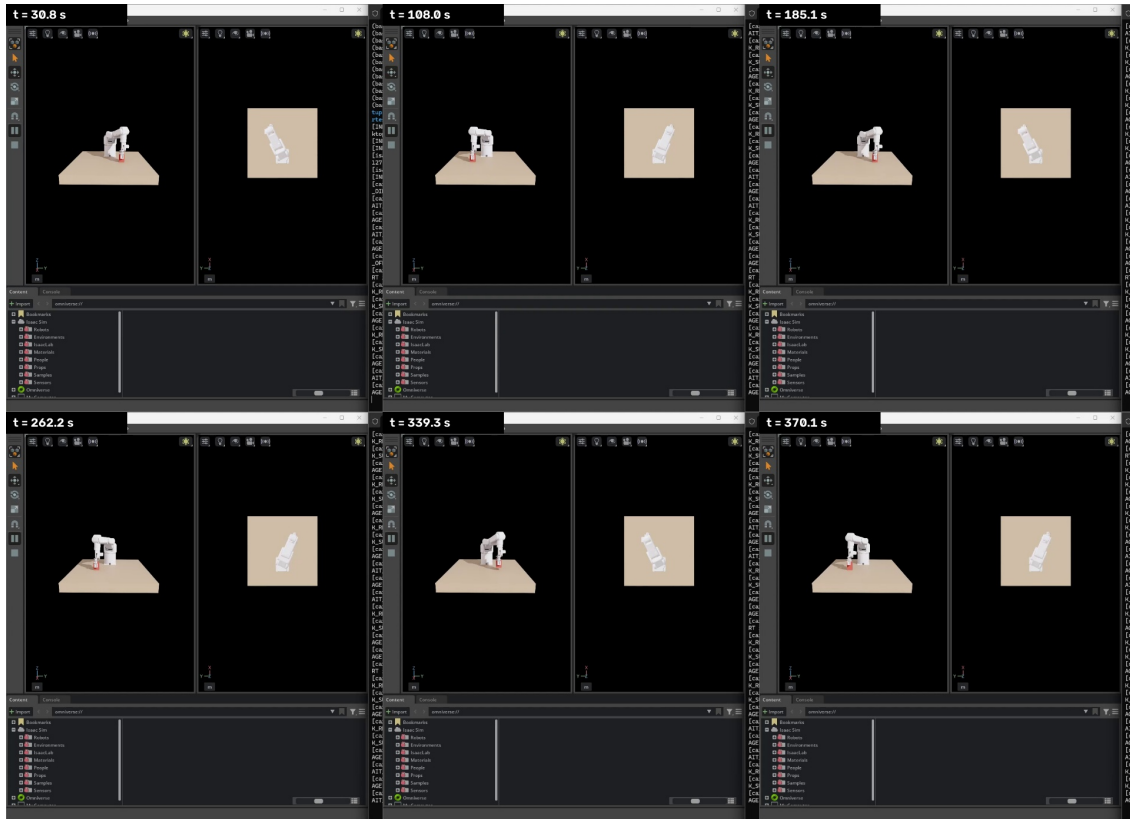


Figure 6 Representative frames from my completed five-cycle Isaac Sim recording

The successful simulation recording is 385.57 s at approximately 30 FPS. It shows the complete alternating sequence with the scene and terminal visible. I also preserved a failed simulation attempt rather than deleting it, because failure evidence is valuable for explaining safe behavior.

3.8 Empty-Grasp and Object Retention Logic

The dedicated empty-grasp program intentionally closes the gripper at a reachable empty point. It emits

Listing 1 Expected controlled empty-grasp output

```
1 EMPTY_GRASP_EXCEPTION code=NO_OBJECT
2 EMPTY_GRASP_RETURN_HOME
```

then reopens, retreats, and returns HOME. This test verifies the exception path without requiring an IK failure.

In simulation, an object-retention monitor checks the distance between the gripper and

target after closure. When the distance is within 0.08 m, it creates a fixed attachment for transport and removes it on release. I used this as a deterministic system-integration aid. It does not replace accurate finger-contact simulation, so I documented the distinction rather than presenting it as a high-fidelity force grasp.

3.9 Physical Implementation on DJI RoboMaster EP

For the real-robot stage, I adapted the fixed-point concept to the degrees of freedom available on the RoboMaster EP. The arm moves between a parked posture $(x, y) = (0, 150)$ mm and a grasp posture $(220, 0)$ mm. The chassis rotates to change the grasp/placement direction. Odd cycles use $+45^\circ, -90^\circ, +45^\circ$ and even cycles reverse all signs, so the object alternates between two fixed sides.

The physical sequence is:

1. park the arm and open the gripper;
2. rotate 45° toward the source;
3. lower and extend the arm;
4. close the gripper and measure the time to the closed state;
5. if closure takes at least 1.85 s, classify it as an empty grasp, retreat, undo the first rotation, and retry;
6. otherwise, park the arm, rotate 90° toward the destination, lower, release, park, and rotate back;
7. count only successful grasps until five successful cycles are complete.

The program subscribes to gripper status at 50 Hz, uses a 5 s timeout for an unknown closure state, starts a 360p camera stream, and monitors the E key. Pressing E stops the chassis, pauses the gripper, suppresses later commands, stops the video stream, unsubscribes status, and closes the robot connection.

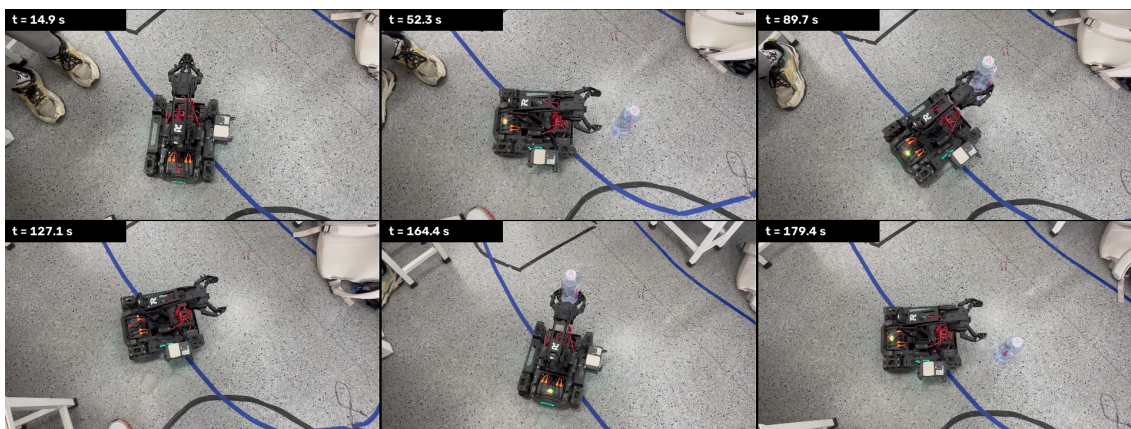


Figure 7 Representative frames from my completed RoboMaster EP five-cycle physical experiment

The physical recording is 186.87 s at 30 FPS. It preserves real approach, grasp, chassis

transfer, release, and return motion, including a failed attempt and subsequent continuation. This is important evidence that the recovery logic was exercised on the actual platform rather than described only in source code.

Chapter 4 Problems, Solutions, and Personal Reflection

4.1 Why I Focused on Root Causes

The most useful lesson from this experiment was that visible robot behavior often has a cause outside the component that appears to fail. A blank RViz window may be a discovery problem, a shaking wrist may be a command-profile or drive problem, and a loose outer jaw may be a missing mechanical constraint rather than an insufficient PID gain. I therefore used a repeated method:

1. reduce the system to the smallest observable path;
2. distinguish command generation, transport, state feedback, and physics;
3. reproduce the symptom with a targeted probe;
4. change one layer at a time;
5. preserve both successful and failed evidence.

4.2 ROS 2 Discovery and Blank RViz

Symptom. Isaac Sim and RViz opened, and `move_group` printed “You can start planning now,” but RViz showed a Global Status warning, no world frame, and no initial planning scene.

Diagnosis. The message only proved that the local MoveIt process had initialized. It did not prove that the other ROS 2 nodes could discover it. Proxy TUN routing and mixed DDS settings caused different processes to use incompatible discovery paths.

Solution. I first repaired the world $\rightarrow$ base static transform and RViz configuration. I then cleared stale discovery variables and tested ROS 2 communication independently. When cross-system DDS remained fragile, I moved the operational ROS graph entirely into the Docker container and used TCP only for the Windows Isaac boundary.

Personal reflection. I learned to treat startup messages as local evidence, not end-to-end evidence. A system is connected only when a command produces motion and the resulting measured state returns through the full path.

4.3 Docker Build and Proxy Failures

Symptom. Docker builds failed first with a non-printable session header and later while fetching the `osrf/ros:humble-desktop-full` token. A recreated container also entered a Restarting (127) loop.

Diagnosis. The project path contained Chinese characters, the Docker Desktop proxy inherited an unsuitable local proxy configuration, and an old image retained a CRLF-affected entrypoint. Recreating the container without rebuilding could not correct the image.

Solution. I staged the Docker build context under an ASCII-only path, configured Docker Desktop's proxy, rebuilt the image, and checked docker compose ps before using exec. I also separated image build, container startup, package build, and task launch in the documentation.

Personal reflection. Reproducibility depends on the host toolchain as much as on source code. I now record the exact shell, current directory, image name, and rebuild conditions instead of writing only the final ROS command.

4.4 Motion Jerk, Shaking, and Over-Damping

Symptom. Early direct targets made the arm move in steps and excite visible oscillation. Increasing damping indiscriminately removed some oscillation but made the arm slow or limp, and gripper motion could disturb the stationary wrist.

Diagnosis. There were three coupled causes: discontinuous target changes, target-stream timing that did not follow feedback, and gains that were not appropriate for both the arm and the gripper loop.

Solution. I introduced quintic 120 Hz interpolation, measured-feedback pacing, and separate normal/vertical speed limits. I then separated arm import drives from the softer gripper drives. When a dynamics change made the arm worse, I rolled it back to the last working baseline before testing another hypothesis.

Personal reflection. I initially treated damping as a single “stability” knob. The experiment showed me that stability is a property of the entire closed loop: trajectory continuity, sample timing, stiffness, damping, inertia, constraints, and collision contacts must be considered together.

4.5 Incorrect Grasp Position and Orientation

Symptom. The arm could move through the nominal sequence, but the object was not between the fingers, and the gripper was not perpendicular to the table during descent. In one later test, applying a 90° yaw target caused numerical IK failure.

Diagnosis. A pose contains both position and orientation. Adjusting only A/B coordinates could not guarantee a vertical tool axis. Conversely, enforcing an orientation at a position near the reach boundary could make the combined target infeasible. The tool frame also differed from my initial mental model, so rotating a joint directly was not equivalent to specifying grasp yaw.

Solution. I derived the target orientation from the HOME tool axis, applied the 90° yaw relative to that axis, and reused it throughout the grasp sequence. I added a dot-product verticality condition and adjusted the reachable pre-grasp height from 0.12 m clearance to 0.10 m. I also moved the closing point upward so the gripper surrounded the block before the palm contacted it.

Personal reflection. This problem changed how I think about robot poses: position and orientation must be solved together, and “looks like a 90-degree rotation” is not a sufficient kinematic specification.

4.6 Missing Gripper Pins and Mimic-Joint Asymmetry

Symptom. The adaptive gripper’s outer links appeared loose, the distal inner and outer links were not locked together, and one side could remain stationary when the master gripper joint moved. Adding only visible pins did not repair the physical behavior.

Diagnosis. The visual meshes showed pivot holes, but the URDF tree omitted the two closed-loop joints. A cylinder mesh has no mechanical effect unless a constraint exists. In addition, PhysX 5.1 mimic behavior was unreliable when several follower joints shared one reference.

Solution. I identified the pivot coordinates from the mesh geometry, created two explicit loop revolute joints, made the outer joints passive, drove the inner joints explicitly, and mirrored the command sign in software. I retained visible pins only as a visual explanation of the constraints.

Personal reflection. This was the clearest example of why a visually complete CAD model is not automatically a physically complete simulation model. I had to reason from the real mechanism’s degrees of freedom and constraint topology.

4.7 Failed Attempts as Engineering Evidence

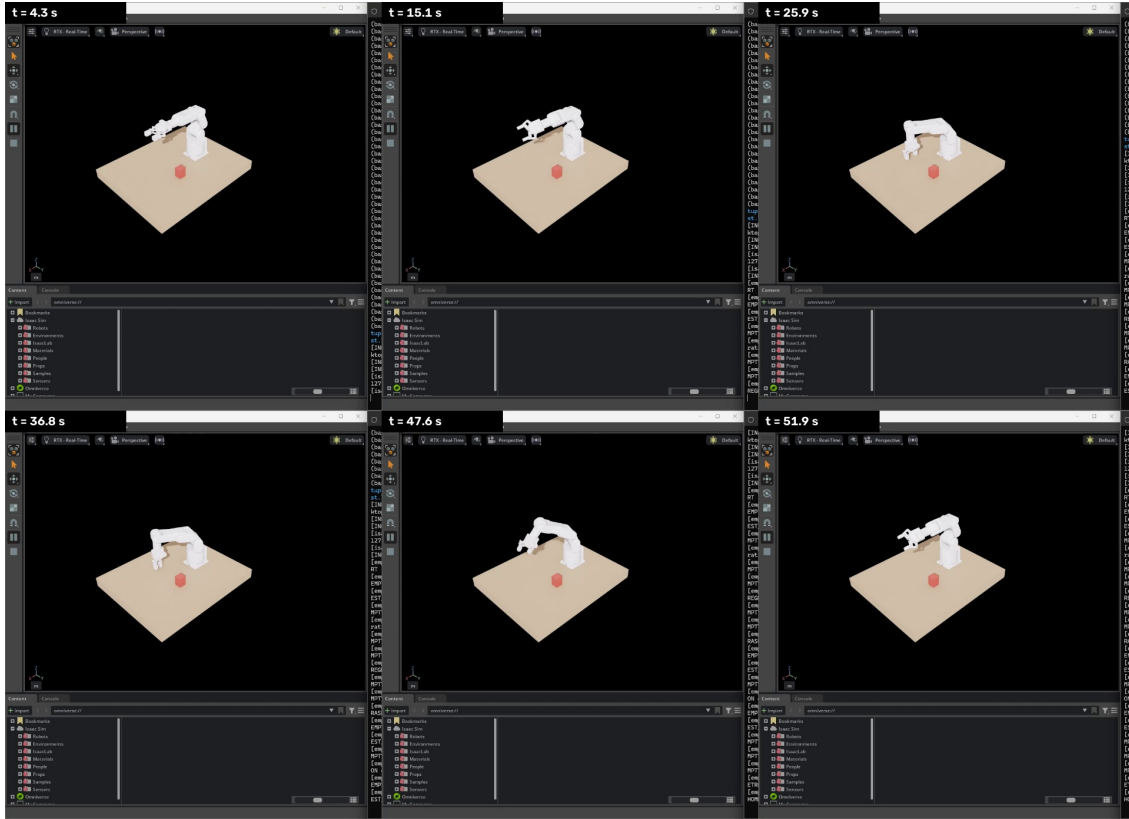


Figure 8 Preserved simulation failure case used to verify visibility and bounded recovery

I preserved the failed simulation recording and implemented an explicit empty-grasp test because a successful video alone cannot demonstrate safe exception behavior. On the physical robot, I used gripper closure time as an available heuristic: a quick closed signal indicates an object obstructed full closure, while a long closure indicates an empty grasp. This is not a calibrated force sensor, but it is a practical observable signal for the available hardware.

The physical program distinguishes three cases:

- closure below 1.85 s: count the grasp and transfer;
- closure at or above 1.85 s: classify as empty, retreat, and retry without increasing the success count;
- no closed state within 5 s: preserve the current state and enter safe exit because the result is unknown.

My main lesson was to represent uncertainty explicitly. A timeout should not be silently converted into success or failure.

4.8 Summary of Major Problems and Decisions

Table 4 Problem-solving summary

Symptom	Root cause	Final decision	What I learned
RViz blank/global warning	DDS discovery and missing fixed frame	Container-local ROS graph, static TF, TCP Isaac boundary	Verify the full data path
Docker build time-out/error	Proxy, Unicode path, stale CRLF image	ASCII staging, proxy correction, clean image rebuild	Host setup is part of reproducibility
Stepped and shaking motion	Target discontinuity and timing/gain mismatch	Quintic interpolation, 120 Hz streaming, feedback pacing	Smooth commands before tuning gains
Slow vertical contact needed	One speed for all stages	28 deg/s transit and 4 deg/s vertical stages	Motion context should select speed
Palm hit before closure	Closing target too low	Dedicated 0.04 m closing clearance	Model the object top, not only its center
IK failed after yaw change	Combined pose near reach boundary	HOME-relative yaw, vertical metric, reachable clearance	Position and orientation are coupled
Outer gripper links loose	Missing closed-loop constraints	Two distal loop revolute joints	Mesh appearance is not physics
One gripper side unresponsive	Multiple-follower mimic limitation	Disable mimic parsing and mirror explicit drives	Middleware/model shortcuts need validation
Empty grasp counted as success	No grasp-result observation	Closure-time heuristic, retry, unknown-state timeout	Uncertainty needs a separate state

4.9 Personal Growth

This project developed four habits that I consider more valuable than any single parameter value.

First, I learned to build a narrow diagnostic probe before debugging the complete task. The `direct_motion_probe` proved outbound motion and return feedback independently of MoveIt planning. That reduced a complicated integration failure to a transport/control question.

Second, I learned to separate model geometry from physical semantics. Repairing the

gripper required locating the correct bodies, local pivot coordinates, joint axes, drive ownership, and collision behavior. It was not enough to add a visible bolt.

Third, I learned to preserve traceability. Small commits, explicit logs, failure recordings, and parameter files made it possible to explain why the final system works and to recover when a tuning change made it worse.

Finally, I learned to adapt one task principle across different robots. The simulated mechArm and physical RoboMaster EP do not share the same kinematics, but both implement fixed source/destination locations, a safe posture, controlled approach, grasp feedback, transport, release, repetition, and recovery. That abstraction is what made the experiment transferable.

Chapter 5 Conclusion

I completed both parts of Experiment 2 with the equipment available to me: a mechArm 270 Pi adaptive-gripper simulation in Isaac Sim and a physical DJI RoboMaster EP experiment. The simulation executes vertical fixed-point grasping, five alternating transfers, and controlled exceptions through a Dockerized ROS 2/TCP architecture. The physical program completes five successful grasp-and-place cycles while monitoring the gripper, retrying empty grasps, displaying the camera, and providing a safe-exit path.

My main contribution was not a single script but the integration of model, physics, kinematics, transport, feedback, real hardware, evidence, and documentation. The public repository history, simulation recordings, failure recording, physical recording, and source cross-reference provide a verifiable account of that work. The project also taught me to distinguish visible symptoms from root causes and to treat failed attempts as useful engineering evidence.

A Repository Evidence Cross-Reference

Table 5 Evidence and implementation paths

Evidence topic	Repository path
Simulation startup and scene	scripts/start_isaac.ps1, simulation/isaac/start_simulation.py, simulation/scenes/mecharm_pick_place.usd
Official model assets	simulation/urdf/mycobot_description/

Evidence topic	Repository path
Cartesian task parameters	src/mecharm_pick_place/config/direct_cartesian_pick_place.yaml
Numerical IK	src/mecharm_pick_place/mecharm_pick_place/numeric_ik.py
Smooth execution	src/mecharm_pick_place/mecharm_pick_place/direct_pick_place_demo.py
Five-cycle task	src/mecharm_pick_place/mecharm_pick_place/cartesian_direct_pick_place.py
Five-cycle launch	src/mecharm_pick_place/launch/alternating_cartesian_pick_place.launch.py
Empty-grasp exception	src/mecharm_pick_place/mecharm_pick_place/empty_grasp_test.py
TCP transport	simulation/isaac/tcp_joint_bridge.py, src/mecharm_pick_place/mecharm_pick_place/tcp_bridge_node.py
Adaptive-gripper repair	simulation/isaac/start_simulation.py
Object retention	simulation/isaac/grasp_monitor.py
RoboMaster EP five-cycle retry	robomaster_ep/scripts/pick_rotate_place_5_cycles_retry.py
Simulation/physical recordings	docs/videos/
Operating instructions	README.md, README_EN.md, robomaster_ep/README.md
Public commit evidence	report/individual_report/evidence/git_history.txt and Figure 1

B Reproducible Operating Commands

B.1 Simulation Startup

From Windows PowerShell, after changing to the Experiment 2 project root:

Listing 2 Start Docker runtime and Isaac Sim

```
1 .\scripts\start_moveit_sim.ps1
```

After the container is running, the ROS 2 workspace build command is:

Listing 3 Build the ROS 2 packages inside Docker

```
1 docker compose exec moveit bash -lc \  
2 "source /opt/ros/humble/setup.bash && \  
3 colcon --log-base /opt/mecharm_ws/log build \  
4 --base-paths /workspace/mecharm_exp2/simulation/urdf/mycobot_description \  
5 /workspace/mecharm_exp2/src \  
6 --build-base /opt/mecharm_ws/build \  
7 --install-base /opt/mecharm_ws/install --symlink-install"
```

The five-cycle task is launched with:

Listing 4 Execute five alternating simulation cycles

```
1 docker compose exec moveit bash -lc \  
2 "source /opt/ros/humble/setup.bash && \  
3 source /opt/mecharm_ws/install/setup.bash && \  
4 ros2 launch mecharm_pick_place \  
5 alternating_cartesian_pick_place.launch.py \  
6 project_root:=/workspace/mecharm_exp2 cycles:=5"
```

B.2 Physical-Robot Startup

After connecting the computer to the RoboMaster EP Wi-Fi and clearing the work area:

Listing 5 Run the physical five-cycle retry program

```
1 conda activate robomaster  
2 $env:RM_SDK_ROOT = "C:\Users\xhr\RoboMaster-SDK-fixed"  
3 python robomaster_ep\scripts\pick_rotate_place_5_cycles_retry.py
```

The operator presses Enter to begin and can press E at any time to request safe exit.

C Evidence Integrity Note

The GitHub screenshot was captured directly from the public main commit page. The 31 pushed commits and their author are also stored as plain-text evidence. The contact sheets were sampled programmatically from the original MP4 recordings without changing the source videos. Their measured metadata is:

Table 6 Recorded experiment evidence

Recording	Duration (s)	FPS	Frames
Successful five-cycle simulation	385.57	30.002	11,568
Preserved simulation failure	54.06	30.004	1,622
RoboMaster EP five-cycle physical run	186.87	30.000	5,606

The ROS 2 workspace build previously completed four packages in 11.6 s. A lightweight source/configuration test run produced 36 passes and two failures. The two failures were stale contract assertions: one expected an older initial gripper-close sequence, and the other expected an earlier $8^\circ/\text{s}$ vertical speed while the implemented value was $4^\circ/\text{s}$. I report these results because transparent evidence is more useful than presenting an artificial all-green test result.

References

- [1] Robotics Systems Integration Laboratory. *Mechanical Arm Fixed-Point Grasping Experiment Requirements*, 2026.
- [2] Elephant Robotics. *mycobot_ros2: myCobot ROS 2 Package, Humble Branch*. https://github.com/elephantrobotics/mycobot_ros2.
- [3] NVIDIA Corporation. *Isaac Sim Documentation, Version 5.1.0*. <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/>.
- [4] Open Robotics. *ROS 2 Humble Documentation*. <https://docs.ros.org/en/humble/>.
- [5] PickNik Robotics. *MoveIt 2 Documentation: Humble*. <https://moveit.picknik.ai/humble/>.
- [6] DJI. *RoboMaster Python SDK Documentation*. <https://robomaster-dev.readthedocs.io/>.